
LAPORAN PRAKTIKUM

TEKNOLOGI IOT II



Disusun oleh:

Nama : Salim Suheri Sitompul

NPM : 25782053

Kelas : TRI 3B

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

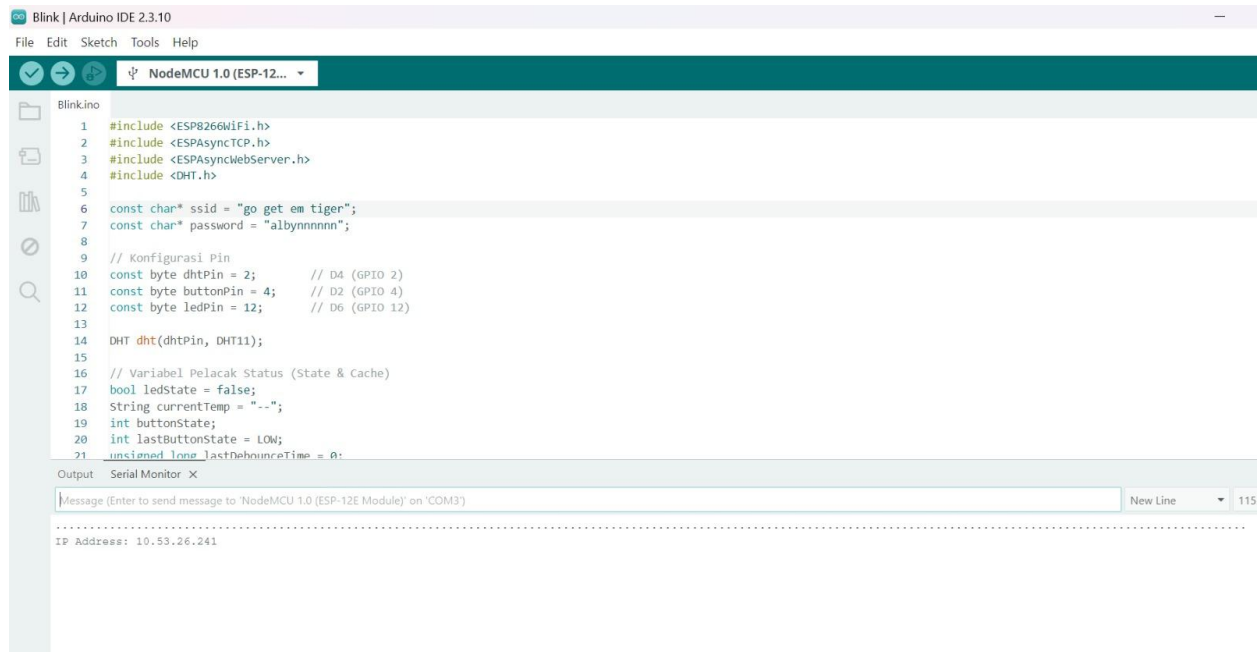
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

Modul 4: Komunikasi Asinkron dan WebSockets

4.5.1 Praktikum 1: Real-Time Dashboard & Sinkronisasi Tombol Fisik

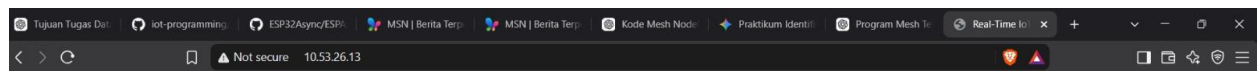


```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "go get em tiger";
7 const char* password = "albynnnnnn";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT11);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

IP Address: 10.53.26.241

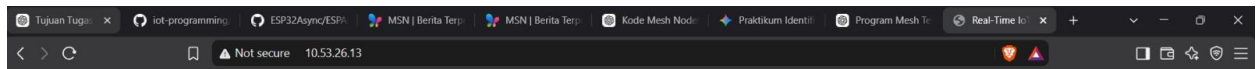


Smart Room

Suhu: 1.70 Celcius

LED: OFF

Turn ON

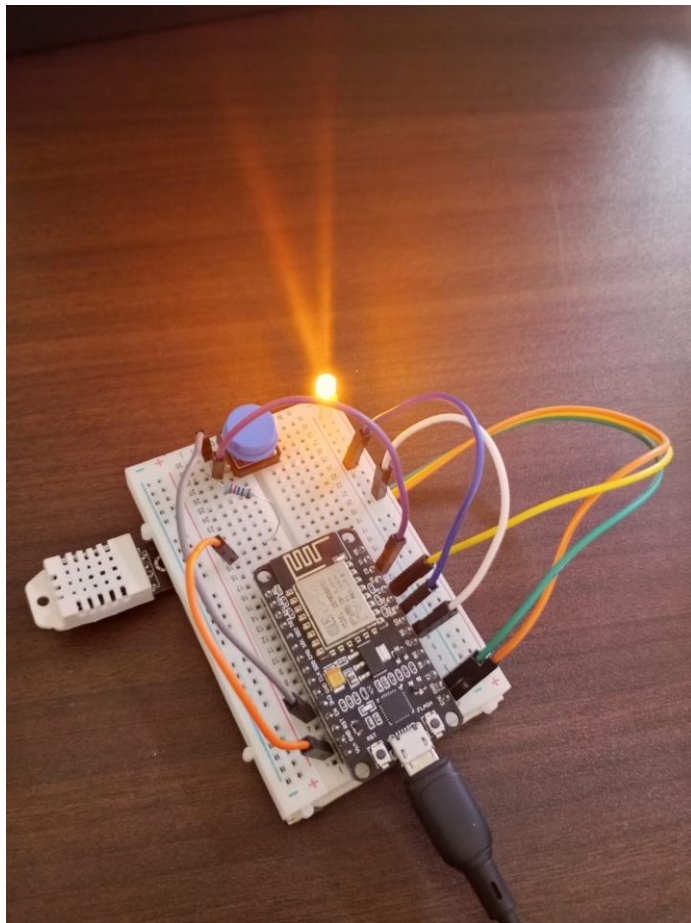


Smart Room

Suhu: 1.70 Celcius

LED: ON

Turn OFF



Langkah Kerja Rangkaian:

1. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin DATA / OUT DHT ke pin D4 (GPIO 2). (VCC DHT ke 3V3, GND ke GND).
3. Pasang Push Button pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur 3V3 (3.3V). Hubungkan kaki seberang tombol ke pin D2 (GPIO 4) NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki Resistor 10k Ohm, lalu hubungkan ujung kaki resistor lainnya ke jalur GND (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).

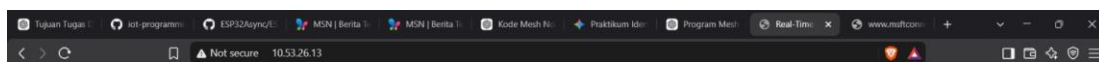
Langkah Kerja Pemrograman: Kode program ini cukup kompleks karena mengombinasikan *Front-End* (HTML, CSS, JavaScript) dengan *Back-End* (C++). Agar *real-time response* tombol tidak terganggu oleh sensor DHT yang butuh jeda baca lama, kita menerapkan teknik pemrograman *non-blocking* menggunakan fungsi `millis()`.

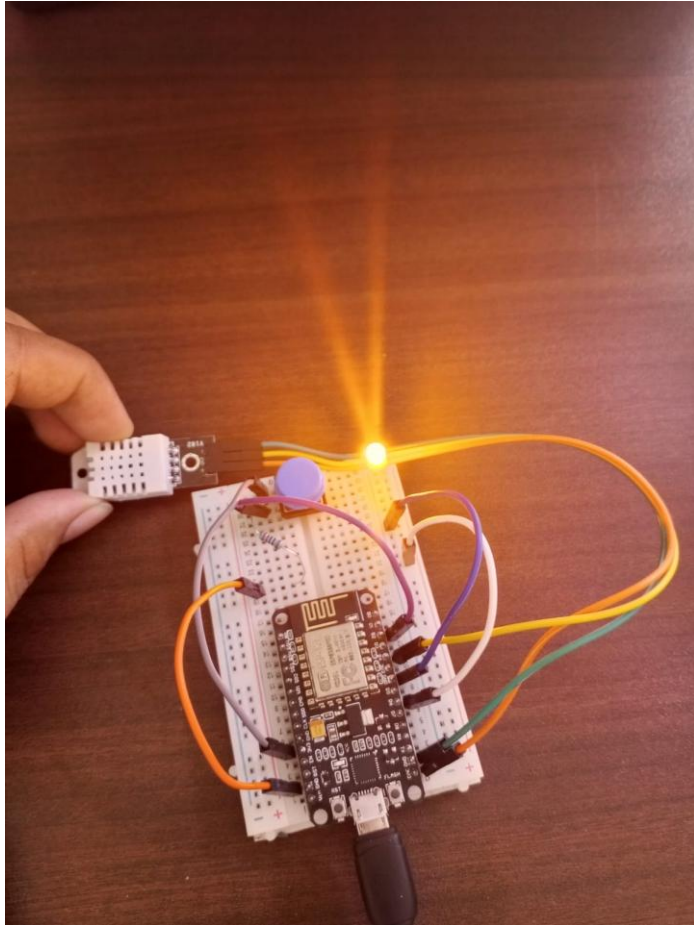
1. Buat berkas baru (*New Sketch*). Tuliskan kode program
2. Upload program. Pastikan IP Address didapatkan dari Serial Monitor.
3. Buka IP Address tersebut di browser laptop Anda, dan sekaligus buka IP yang sama di browser ponsel (smartphone) Anda secara berdampingan.
4. Uji 1 (Kendali Web Tersinkronisasi): Klik tombol pada browser laptop Anda. Perhatikan bagaimana browser di ponsel Anda ikut berubah (Turn ON) secara presisi, dan lampu LED fisik ikut menyala!
5. Uji 2 (Kendali Perangkat Keras): Sentuh push button mekanik di meja Anda. Amati bahwa antarmuka browser di laptop maupun ponsel Anda secara serentak berganti status (Turn OFF menjadi Turn ON) tanpa terjadi proses loading atau refresh HTML halaman sama sekali! Kecepatan respon seketika ini adalah manfaat dari pemrograman Asynchronous dan pemisahan (decoupling) pembacaan memori sensor DHT.

Penjelasan Singkat Kode Khusus:

- `ws.cleanupClients()`: (Berada di siklus `loop()`). Fungsi penting dari pustaka *Asynchronous* ini bertugas membersihkan dan menutup sesi (*resource memori*) dari klien/browser yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (*memory leak*) dan berujung pada *Crash/Restart* jika terlalu banyak perangkat keluar-masuk membuka halaman *Dashboard*.
- `pinMode(buttonPin, INPUT)` dan `if (buttonState == HIGH)`: Mempertahankan konfigurasi resistor *Pull-Down* eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika **LOW** (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika **HIGH** yang memicu pembalikan status (*toggle*) LED secara instan.
- `if ((millis() - lastTime) > 3000)`: Pemrograman *non-blocking*. Alih-alih membuat CPU tidur selama 3 detik menggunakan `delay(3000)` (yang akan membekukan respons WebSocket), kita menggunakan selisih penghitung jam internal CPU (`millis()`). Dengan demikian, CPU dapat terus siaga mengawasi sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.

- ## 4.6 Latihan





Latihan 1 – Teori Jaringan

Penjelasan: WebSockets lebih hemat bandwidth karena koneksi tetap terbuka dan data dikirim tanpa melakukan refresh halaman berulang. Tampilan juga tidak berkedip karena halaman tidak dimuat ulang.

Langkah: Buat koneksi WebSocket → kirim data dari NodeMCU → terima data di browser → tampilkan tanpa refresh.

Latihan 2 – Analisis Timer

Penjelasan: `delay(3000)` menghentikan proses NodeMCU selama 3 detik sehingga komunikasi WebSocket dapat terlambat atau terganggu.

Langkah: Hindari `delay()` → gunakan `millis()` → komunikasi WebSocket tetap berjalan.

Latihan 3 – Modifikasi Front-End

Penjelasan: JavaScript menggunakan `document.getElementById()` untuk mengubah teks pada elemen `<span>` sesuai data dari NodeMCU.

Langkah: Cari `getElementById()` pada `onMessage()` → catat ID elemen yang diubah → gunakan ID tersebut untuk menampilkan data.

Contoh:

```
ledState  
tempValue  
humValue
```

Latihan 4 – Telemetry Kelembapan

Penjelasan: Sistem dimodifikasi agar dapat membaca dan menampilkan kelembapan secara real-time melalui WebSockets.

Langkah:

```
String currentHum = "--";
```

Baca sensor:

```
currentHum = String(dht.readHumidity());
```

Tambahkan ke JSON:

```
{"led": "...", "suhu": "...", "hum": "..."}"
```

```
<h2>Kelembapan: <span id="humValue">--</span> %</h2>
```

```
document.getElementById("humValue").textContent = data.hum;
```

Hasil: Data suhu dan kelembapan dapat ditampilkan secara real-time tanpa refresh halaman.